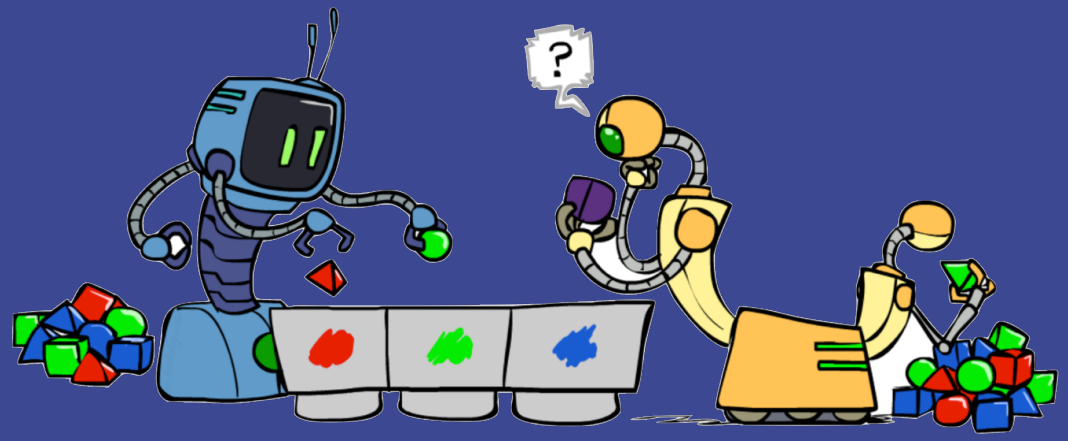
CS 156
Intro to AI

Nearest Neighbor Clustering



These slides are based on the slides created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.
The artwork is by Ketrina Yim

Today

- Nearest Neighbor Classification / Similarity
- Formalizing Learning
- Homework 7
- Unsupervized Learning:
  - Clustering: K-Means

Nearest-Neighbor Classification

- Nearest neighbor for digits:
  - Take new image
  - Compare to all training images
  - Assign based on closest example


0


1


2


0


1


2

- Encoding: image is vector of intensities (features are pixel values)

What's the similarity function?

Basic Similarity

- Many similarities based on the feature vectors dot products:

  $\text{sim}(x, x') = f(x) \cdot f(x') = \sum_i f_i(x) f_i(x')$

- Feature vectors must be normalized first
- We can also base the similarity on the Euclidian distance (in the feature space).
- Note: not all similarities are of this form

Invariant Metrics

- Better similarity functions use domain knowledge
- Computer Vision example: invariant metrics:
  - Similarities are invariant under certain transformations
  - Rotation, scaling, translation, stroke-thickness…
  - Example:

  - Challenge: how can we incorporate such invariances into our similarities?

iClicker

Which classification is faster?

A. Perceptron

B. Nearest neighbor

iClicker

Which classification is faster?

A. Perceptron

B. Nearest neighbor

Lengthy searches for nearest neighbors.

For each prediction, we have to search through the whole training data set.

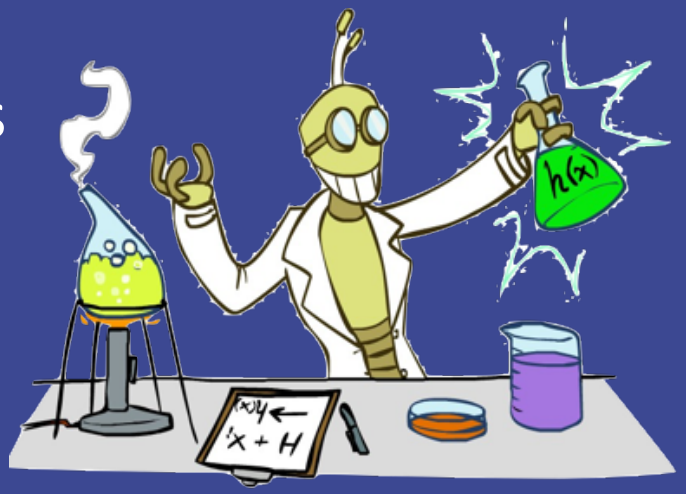
A Tale of Two Approaches…

- Nearest neighbor-like approaches
  - Can use fancy similarity functions
  - Don’t actually get to do explicit learning

- Perceptron-like approaches
  - Explicit training to reduce error
  - Usually linear classification
  - Faster classification

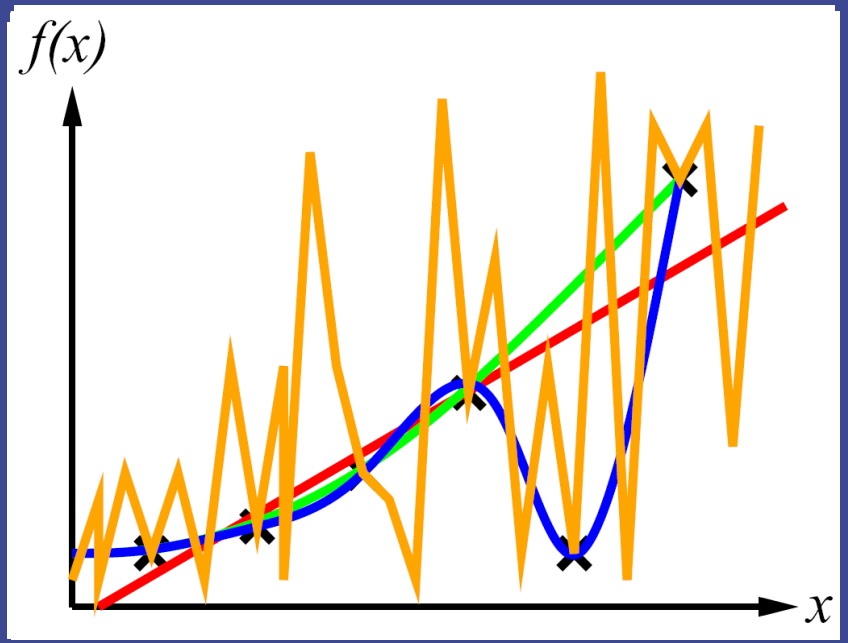
Formalizing Learning

- Inductive learning (discovery learning) is a process where the learner discovers rules by observing examples.
- These rules are represented a function *f,* unknown to us
  - Training examples come in pairs: (x, *f*(x))
  - Example: x is an email, *f*(x) is Ham or Spam
  - Example: x is a house and *f*(x) is its price
- Goal:
  - Given a training data set, discover the best hypothesis *h* that approximates the true function *f* best.
- Includes:
  - Classification: *f* (x) is in a finite set of values
  - Regression: *f* (x) can be any number



Inductive Learning

- Construct/adjust *h* to agree with *f* on training set
- *h* is consistent if it agrees with *f* on all examples
- curve fitting (regression)



Consistency vs. simplicity

The blue and orange curves are consistent.

The red and green ones are simpler.

Consistency vs Simplicity

Occam's razor: simpler is better

"Simpler" solutions generalize better – avoid overfitting

Several ways to "simplify":

- Reduce the hypothesis space
  - Assume more: independence assumptions, as in naïve Bayes
  - Have fewer (better) features
- Regularization
  - Smoothing

Homework 7 – Iris Classification

Iris-setosa

File:Kosaciec szczecinkowaty Iris setosa.jpg

Iris-versicolor

Blue flag flower close-up (Iris versicolor). Photo taken by Danielle Langlois in July 2005 at the Forillon National Park of Canada, Quebec, Canada.

Iris-virginica

Frank Mayfield - originally posted to Flickr as Iris virginica shrevei BLUE FLAG image of Iris virginica shrevei BLUE FLAG at the James Woodworth Prairie Preserve - a bud and a single flower at full bloom

Homework 7: Iris Dataset

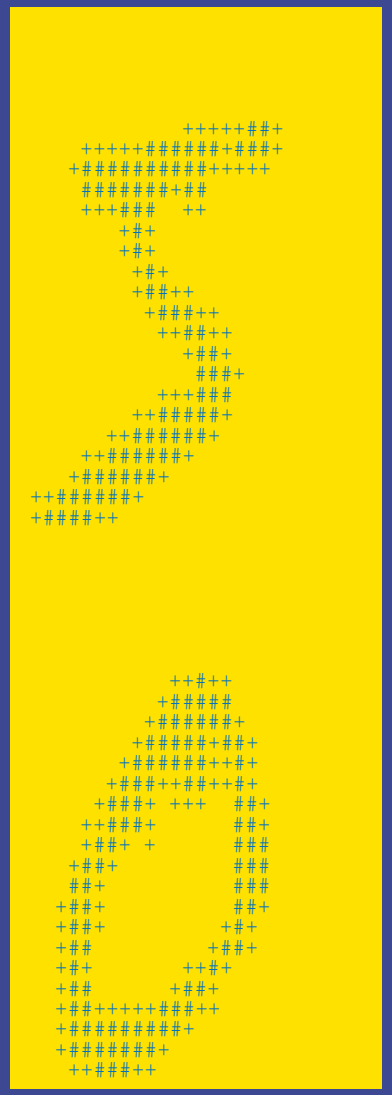
train.csv

```
7,3.2,4.7,1.4,Iris-versicolor
6.7,3,5,1.7,Iris-versicolor
6.5,3,5.2,2,Iris-virginica
...
```

```
def read_iris(filename):
    """
    Read the Iris csv file and construct the Example objects.
    The file is assumed to contain 5 columns:
    sepal length, sepal width, petal length, petal width and training label.
    The first 4 columns will be used as features in the feature vector
    constructed.
    :param filename: Name of the csv file containing iris data
    :return: A list of Example objects
    """
```

Homework 7: Digit Recognition

trainimages.txt



trainlabels.txt

```
5
0
…
```

The image is assumed to be 28 x 28 pixels.
Each pixel represents a feature.

Homework 7 – Example Class

```
class Example:
    """
    Represent an example (training, valid
    Arguments:
        label: (may be integer or string) training label
        fvector: (NumPy array)) feature vector - defaults to None

    Attributes:
        label: (may be integer or string) training label
        fvector: (NumPy array) feature vector
    """
```

Feature vectors are represented as NumPy arrays.

Homework 7 – Example Class

```
class Example:


def distance(self, other):
    """
    Compute the Euclidean distance between the two feature
    vectors representing the given examples.
    :param other: (Example object)
    :return: float
    """
    diff = (self.fvector - other.fvector) ** 2
    return diff.sum() ** 0.5
```

To compute the distance between two examples:
example1.distance(example2)

Homework 7: Multiclass Perceptron

Implement 2 methods:

```
def update_weights(self, example):
    """
    Update the Perceptron weights based on a single training example
    :param example (Example): representing a single training example
    :return: None
    """
    # Enter your code and remove the statement below
    return NotImplemented
```

Homework 7: Multiclass Perceptron

Implement 2 methods:

```
def predict(self, example):
    """
    Predict the label of the given example
    :param example (Example): representing a single example
    :return: label: A valid label
    """
    # Enter your code and remove the statement below
    return NotImplemented
```

Homework 7: knn

Implement 1 function:

```
def predict_knn(data, example, k):
    """
    Classify an example based on its nearest neighbors
    :param data: list of training examples
    :param example (Example o
    :param k: number of neares
    :return: label: valid label fro
    """
    # Enter your code and remo
    return NotImplemented
```

Make sure you don't modify the list argument *data*.
To make a copy of the list that you can modify, you can use a slice assignment:
working_data = data[:]

Homework 7: knn

Implement 1 function:

def predict_knn(data, example, k):

1. Make a working copy (not an alias) of the data list.
2. Initialize an empty list to contain the k neighbors
3. Create a loop (with k iterations) where you
   - find the closest example to the example you are classifying. You do that using the built-in function *min* and a lambda function for the key
   - remove that closest example from the working list and add it to to the neighbors list
4. Make your prediction based on the examples in the neighbors list.

    return NotImplemented

Hint: review the voting example in the CS 156 Python Essentials tutorial.

Homework 7

We'll use the validation data to tune the hyperparameter

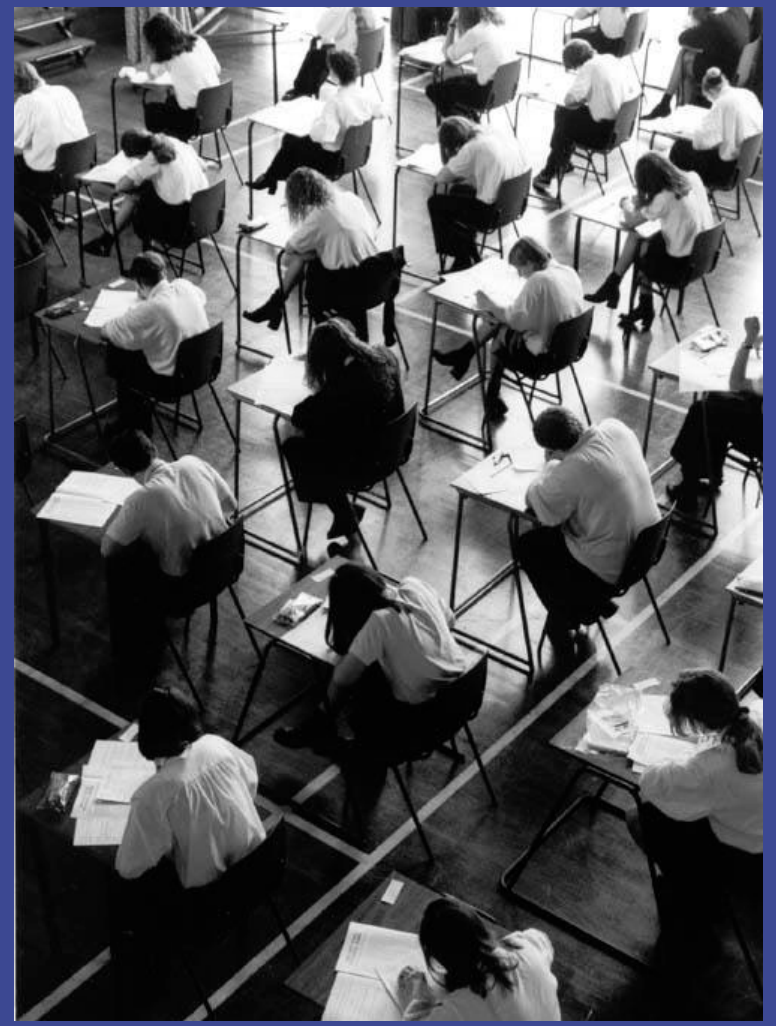
- Run the classifier for several values of the hyperparameter
- Pick the value that gives us the highest accuracy on the validation data

Hyperparameter:

- perceptron: number of iterations
- knn: k (number of neighbors)

Recap: Classification

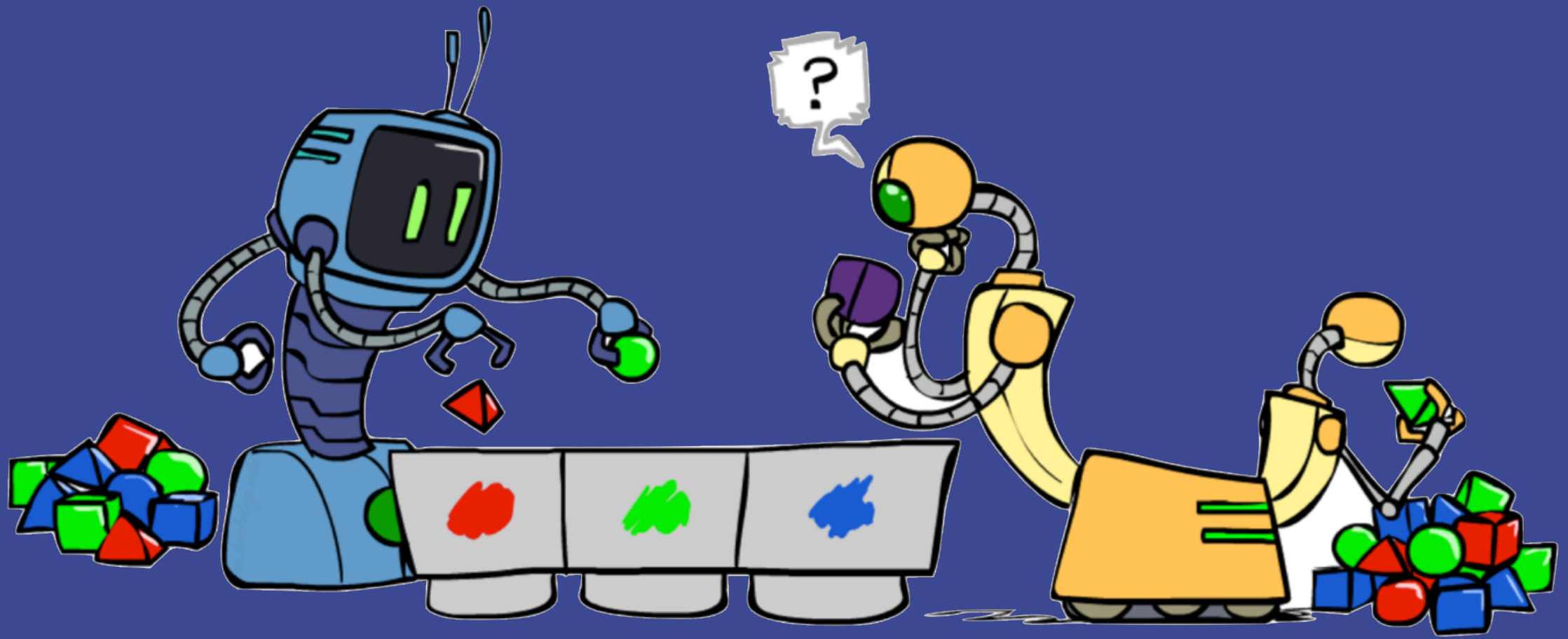
- Supervised learning
- We've seen several methods for this
- Make a prediction given evidence
- Useful when we have labeled data



Clustering

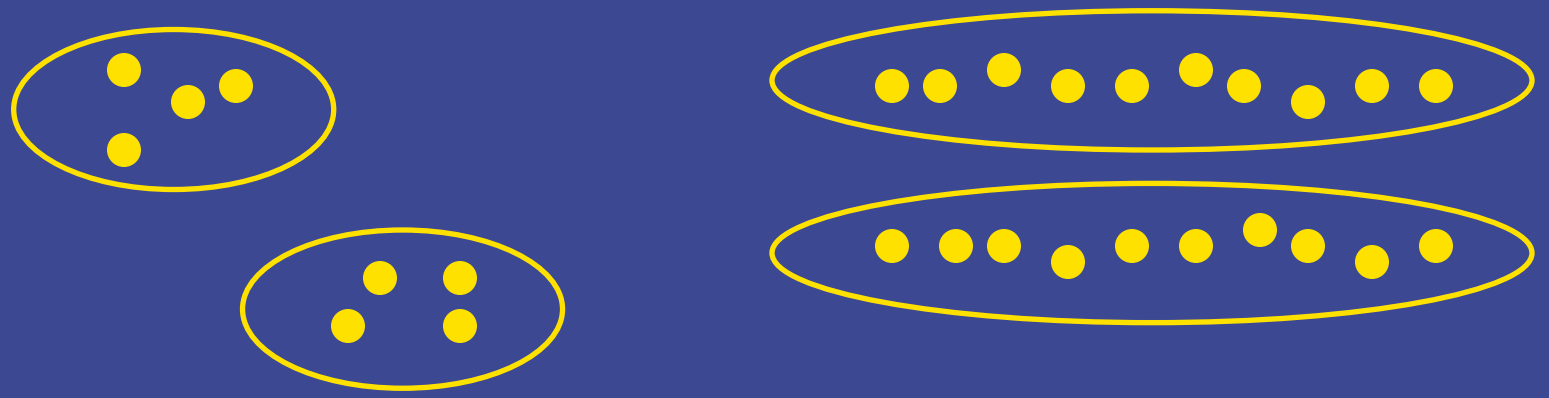
- Unsupervised learning
- Detect patterns in unlabeled data
  - group emails, search results, images
  - find categories of customers
  - detect anomalous/malicious program executions
- Useful when we don't know what we're looking for
- Requires data, but no labels
- Often get gibberish

Clustering



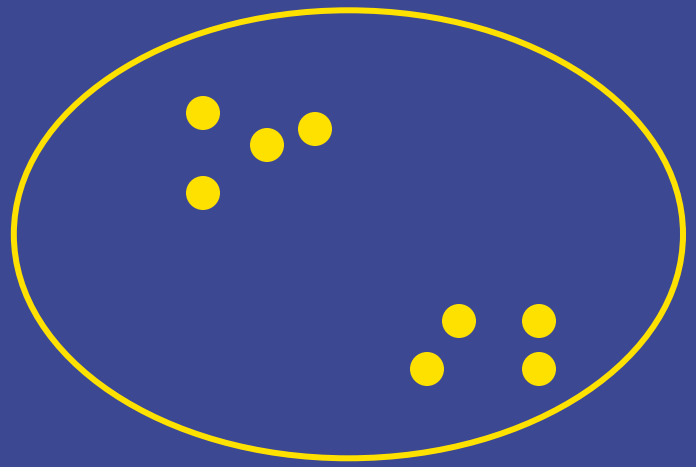
Clustering

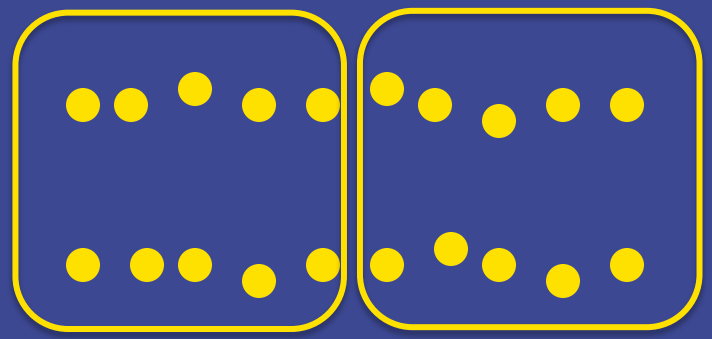
- Basic idea: group together similar instances
- Example: 2D point patterns



Clustering

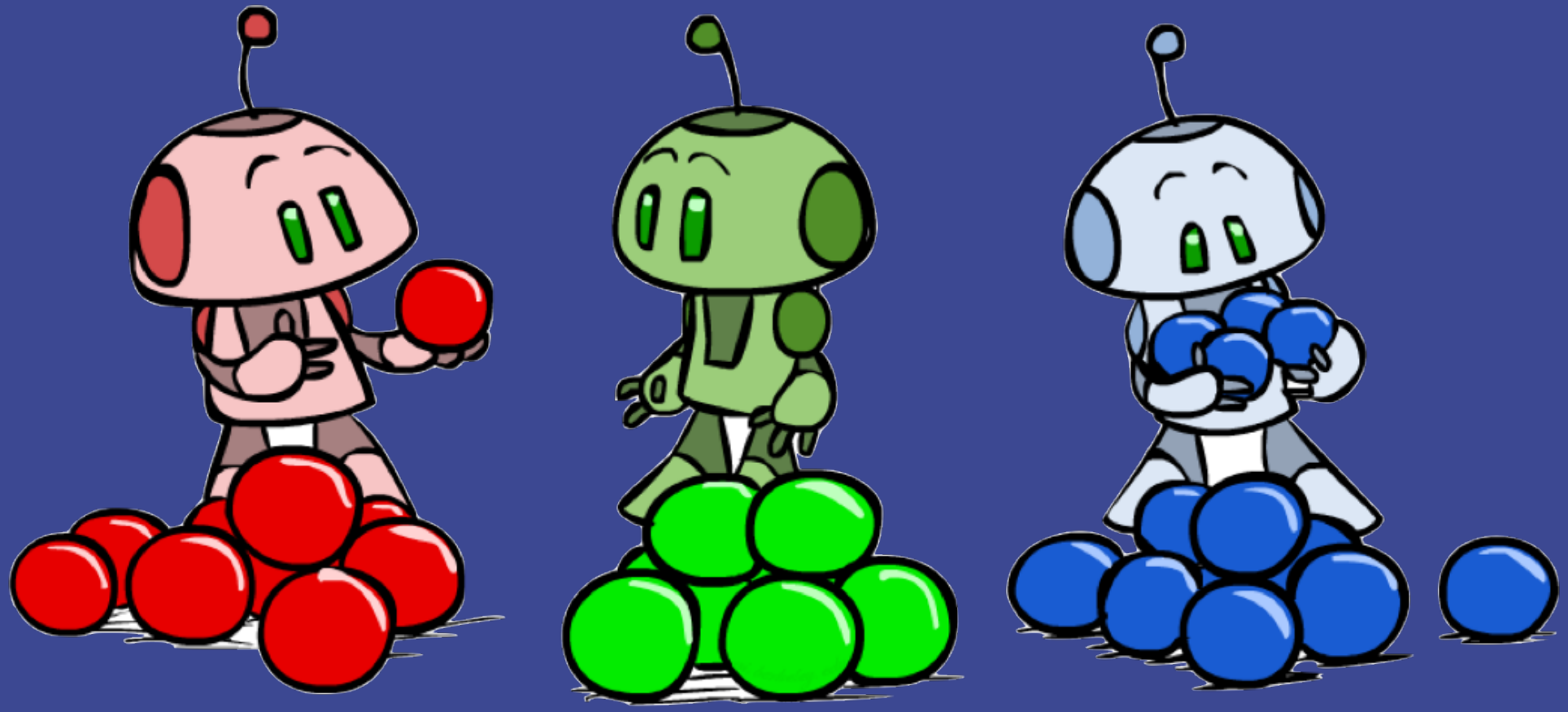
- Basic idea: group together similar instances
- Example: 2D point patterns





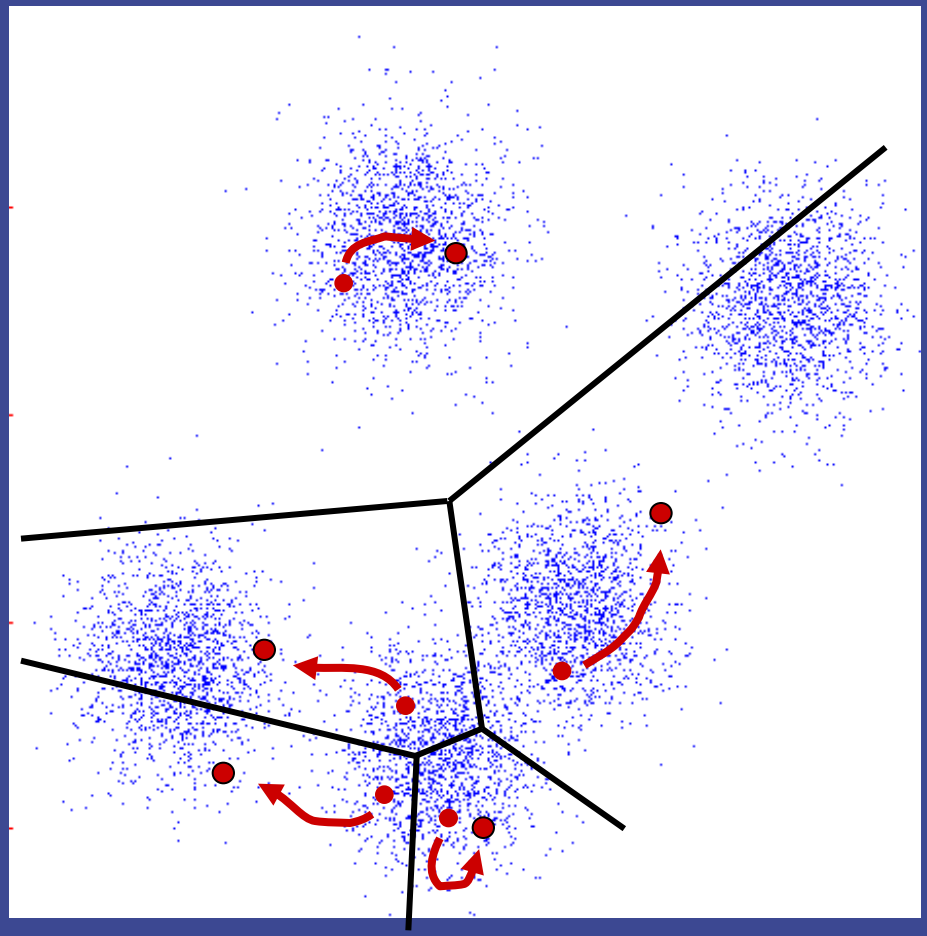
- What could "similar" mean?
  - One option: small (squared) Euclidean distance

K-Means



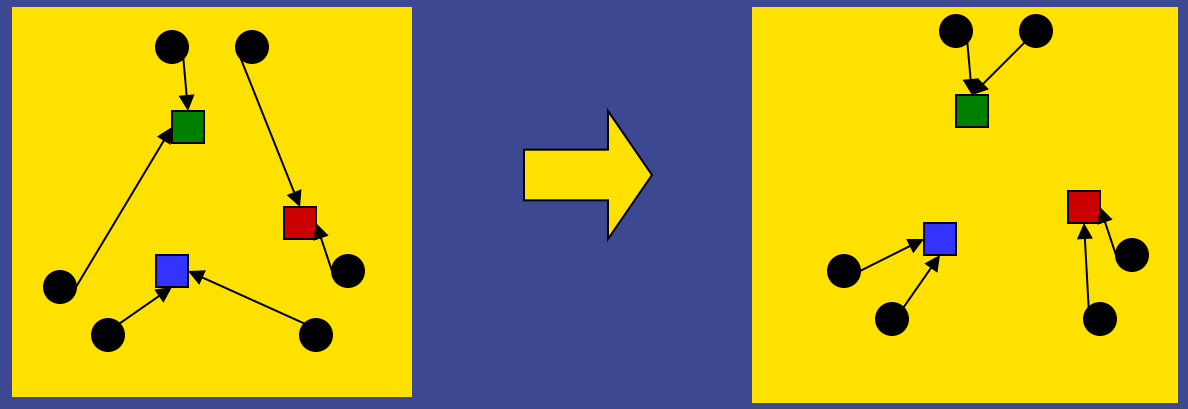
K-Means: An iterative clustering algorithm

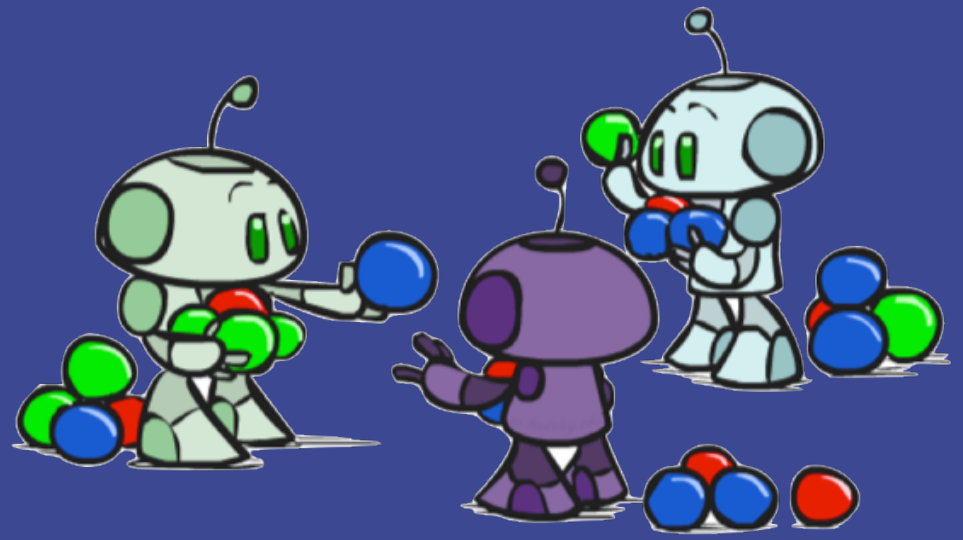
- Pick K random points as cluster centers (means)
- Repeat:
  - Assign data instances to closest mean
  - Assign each mean to the average of its assigned points
- Stop when no points' assignments change



Phase I: Update Assignments

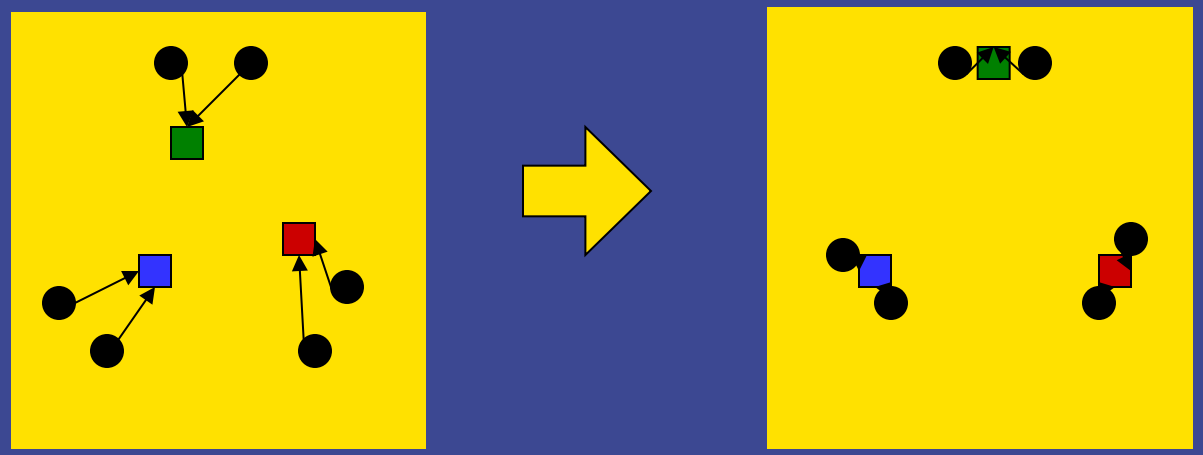
- For each point, assign to closest mean:

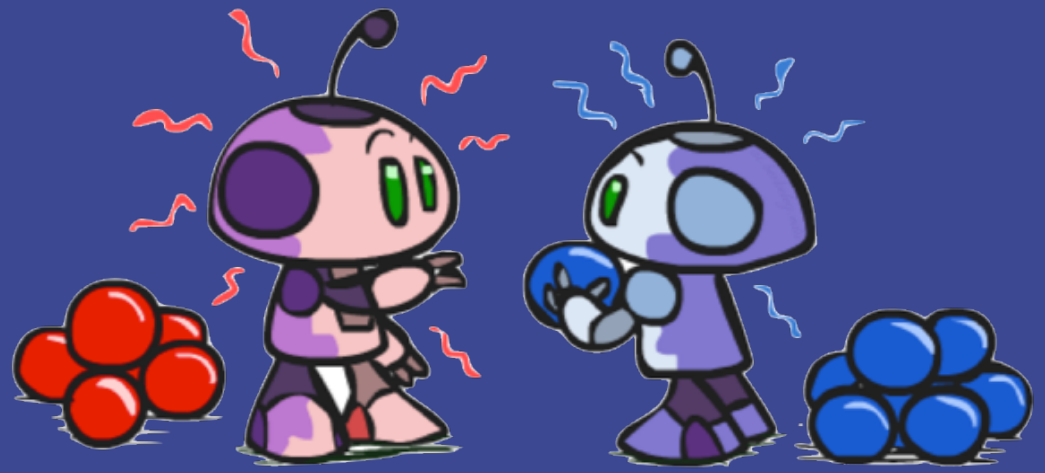




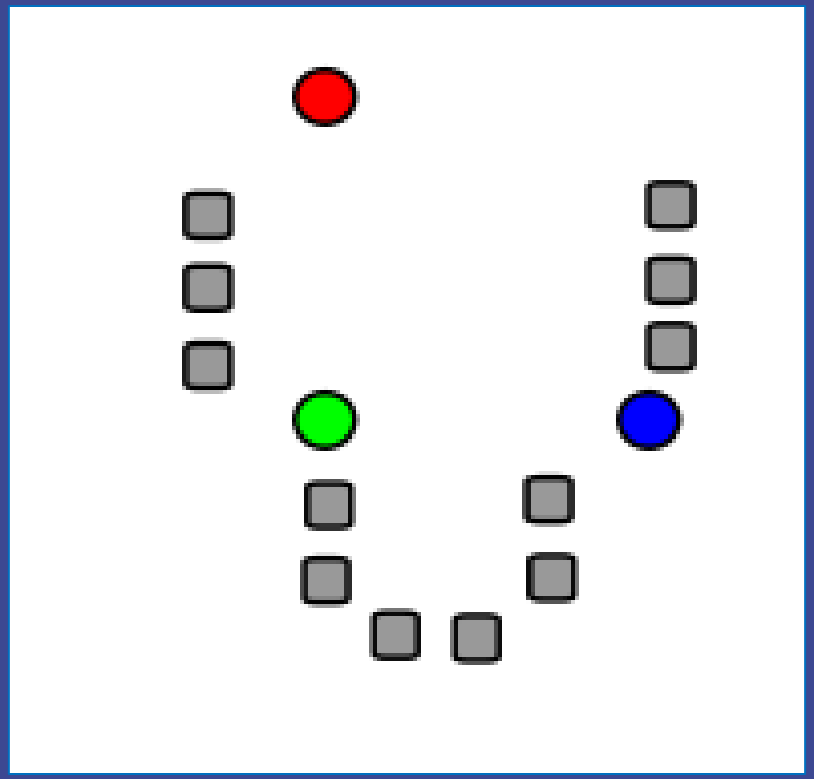
Phase II: Update Means

- Move each mean to the average of its assigned points:

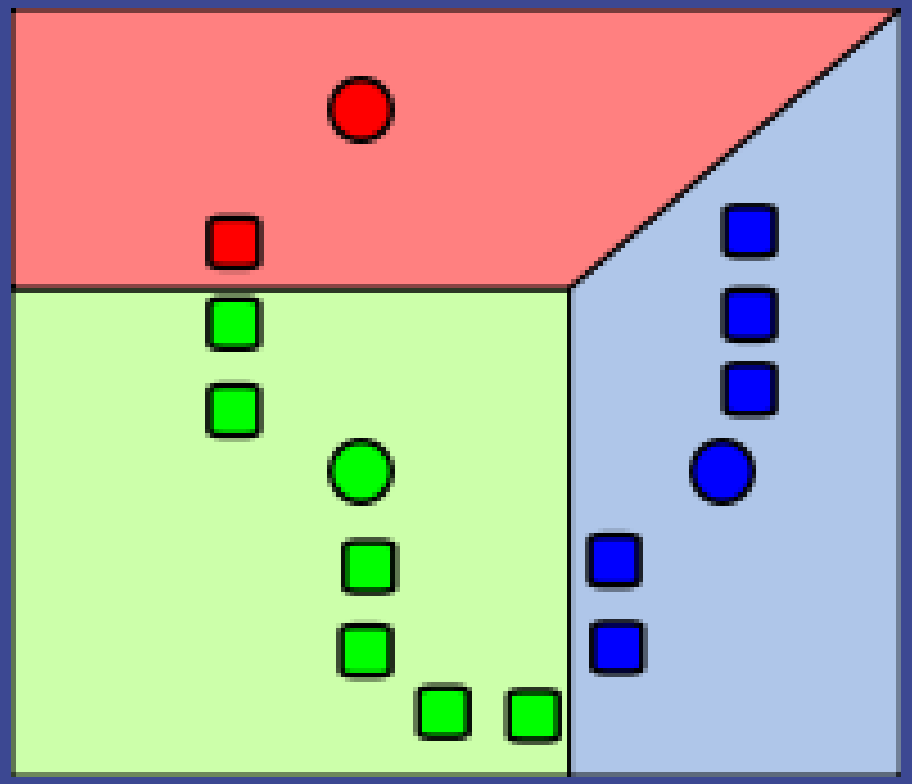




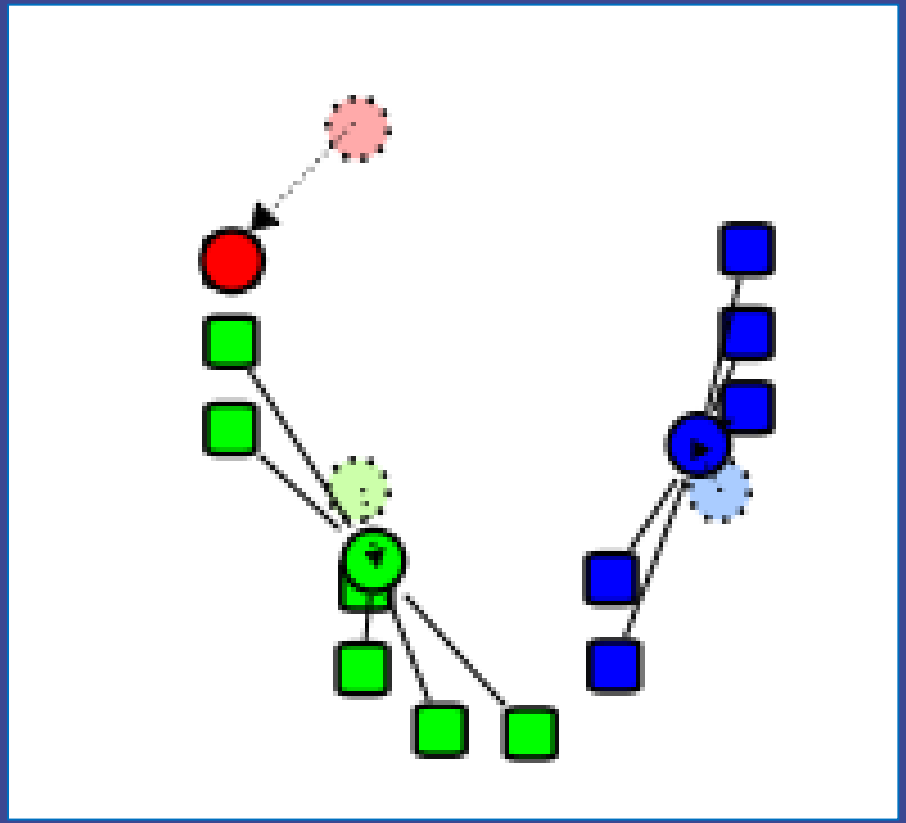
K-Means Example



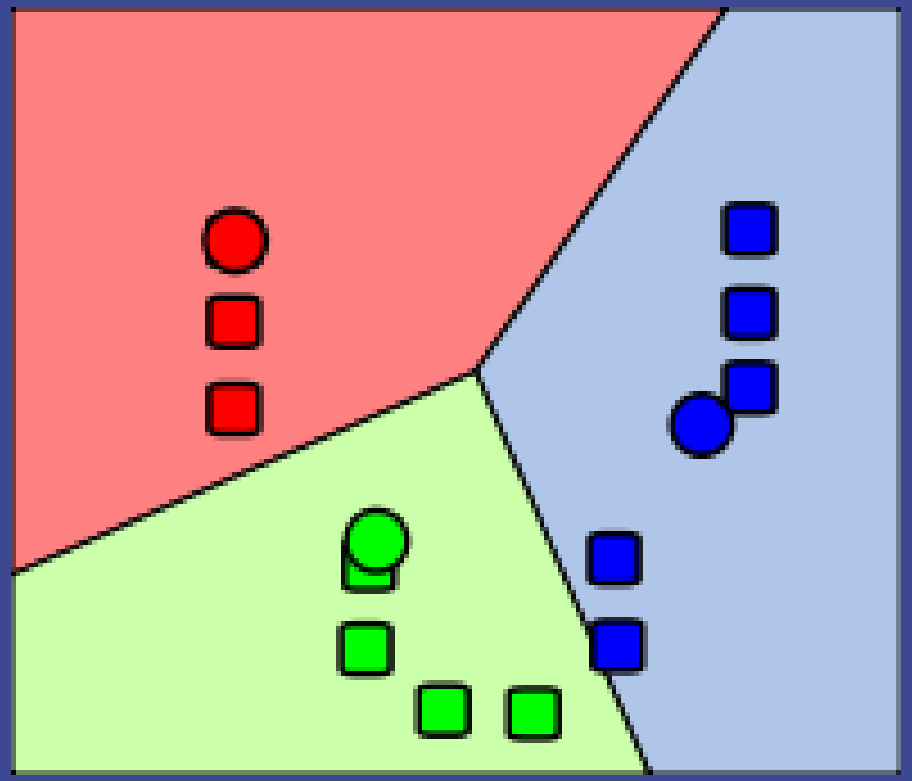
K-Means Example



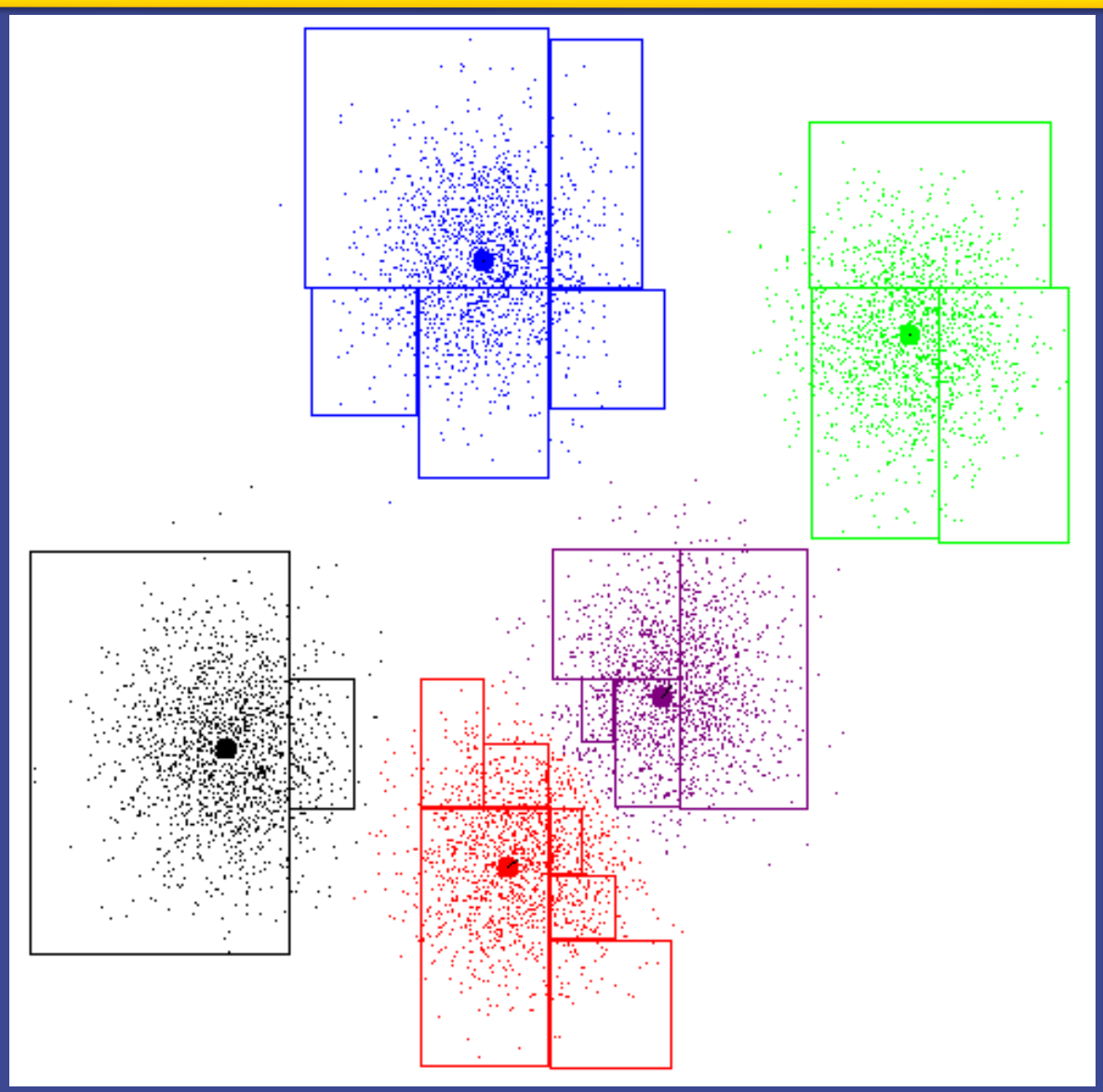
K-Means Example



K-Means Example

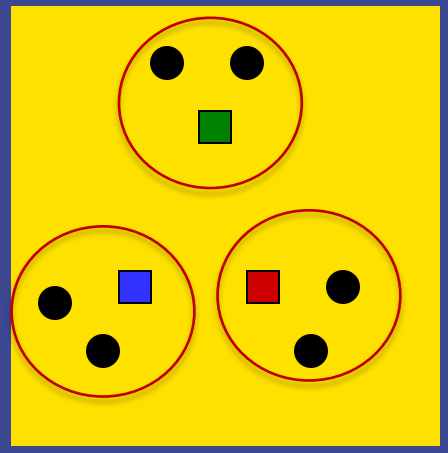


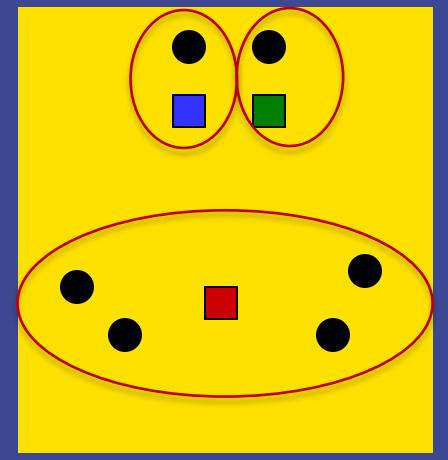
K-Means Example

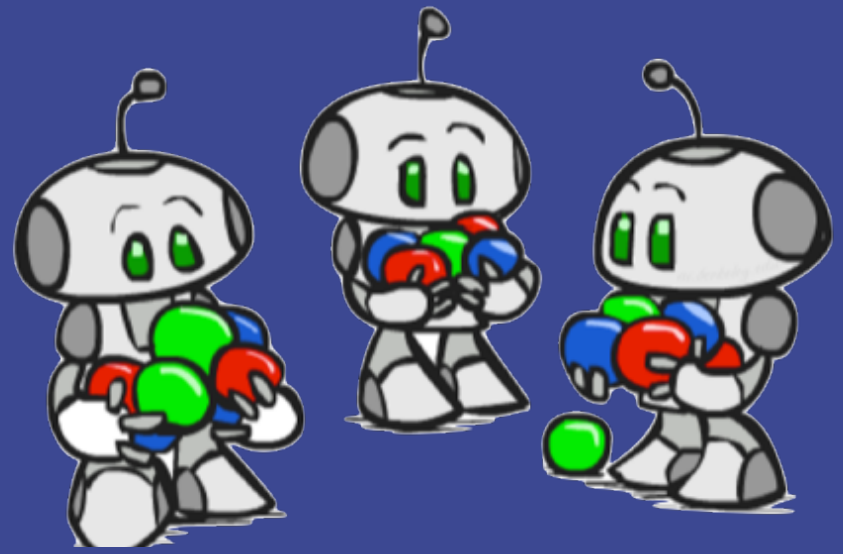


Initialization

- K-means is non-deterministic
  - Requires initial means
  - It does matter what you pick!
  - What can go wrong?
  - Various schemes for preventing this: initialization heuristics, …

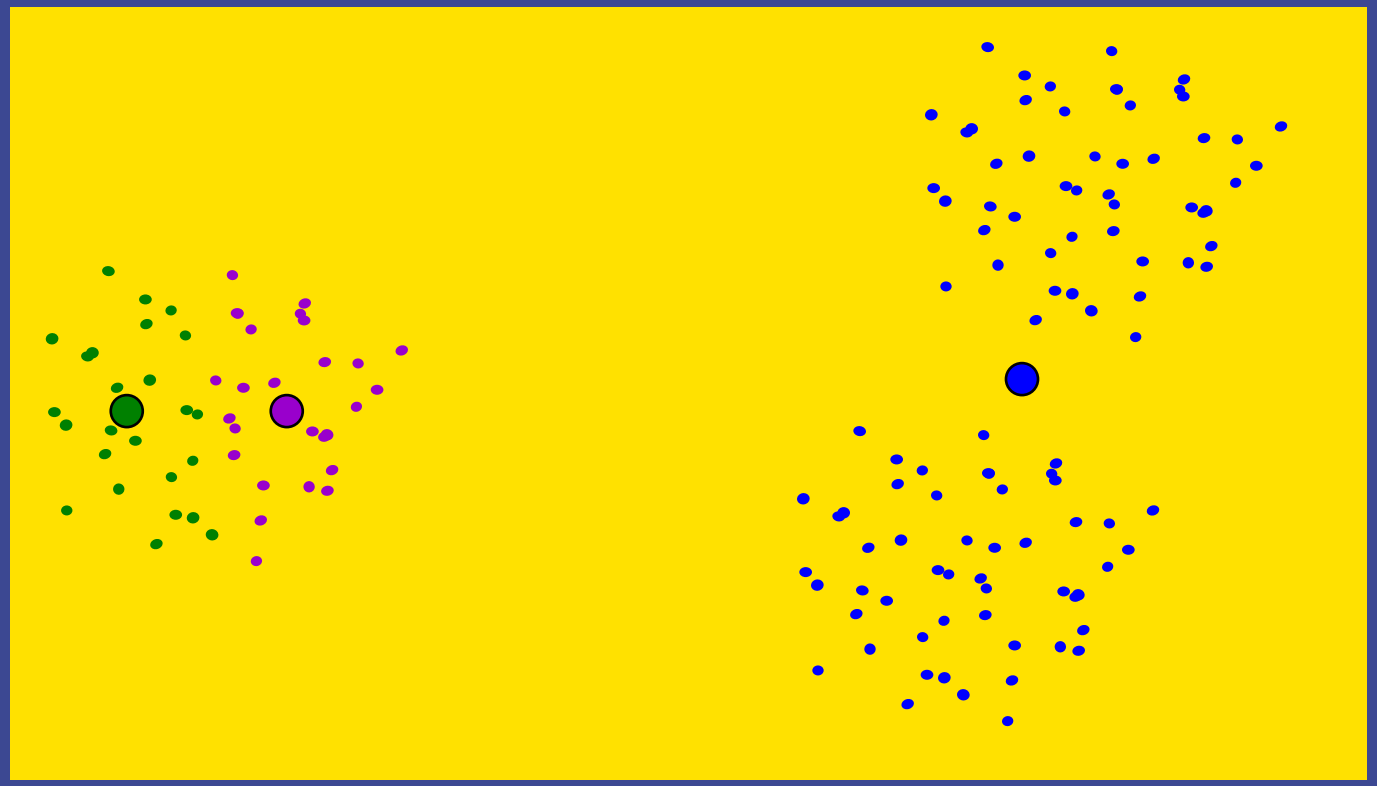




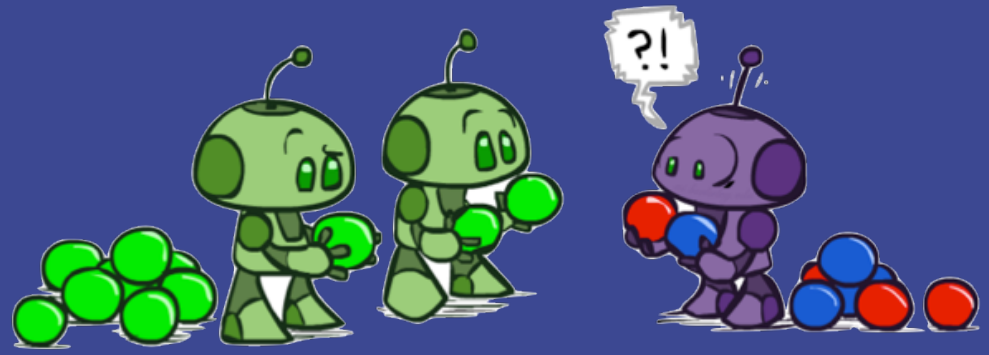


K-Means Getting Stuck

- A local optimum:

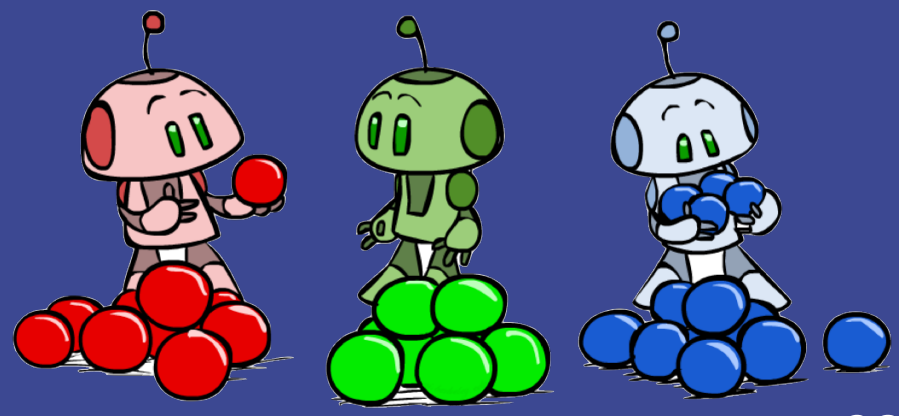


Why doesn't this work out like the earlier example?



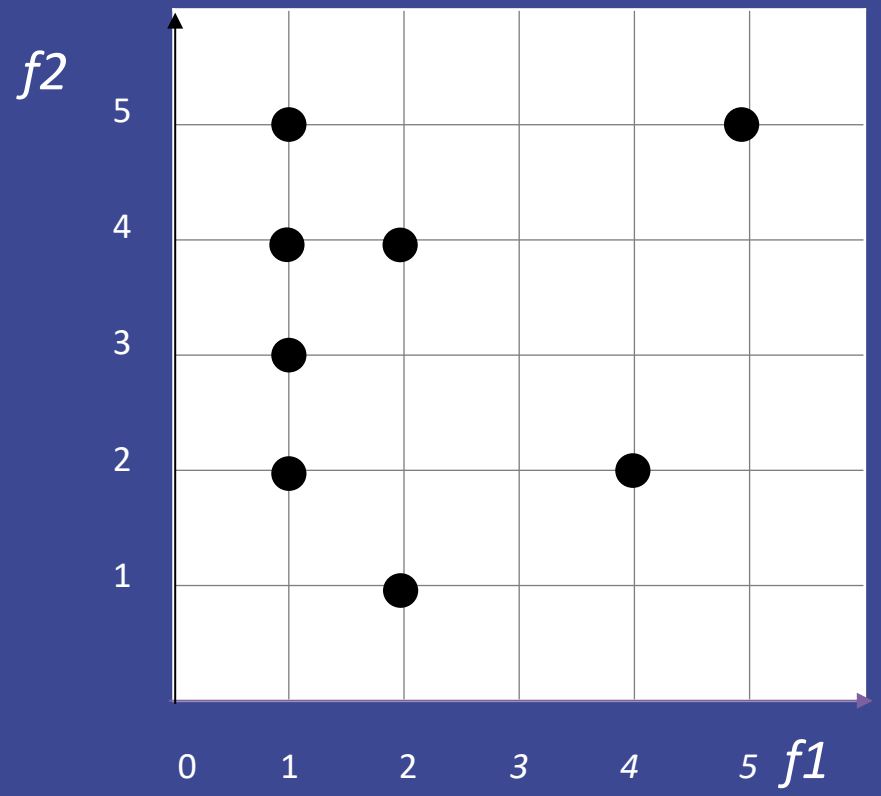
K-Means Questions

- Will K-means converge?
  - To a global optimum?
- Will it always find the true patterns in the data?
  - If the patterns are very very clear?
- Will it find something interesting?
- Do people ever use it?
- How many clusters to pick?



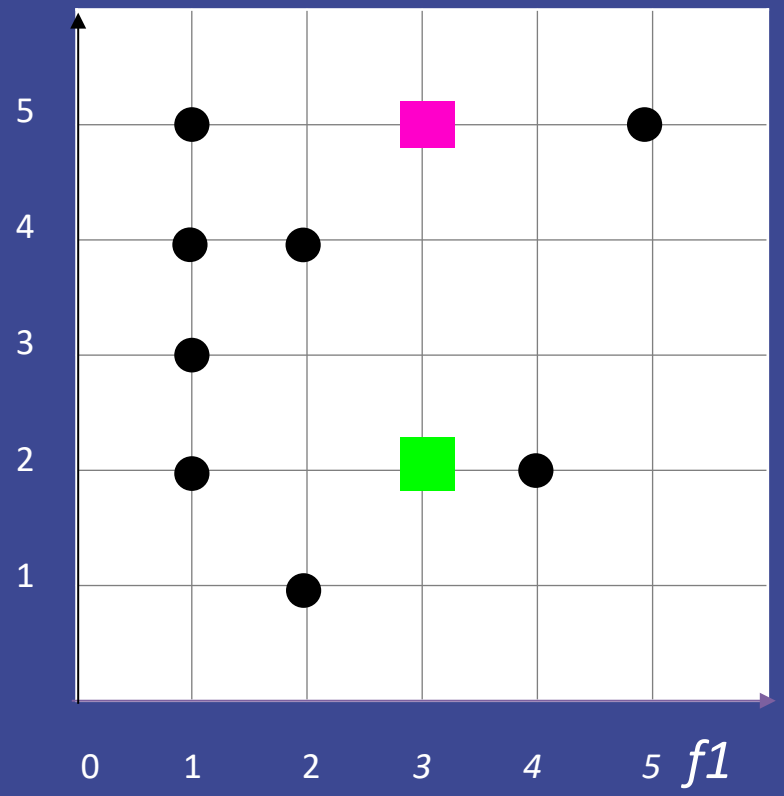
K-Means Clustering

Consider the unlabeled data shown in two dimensional feature space.
Our goal is to apply the k-means algorithm with k=2 to identify clusters in the data.



K-Means Clustering

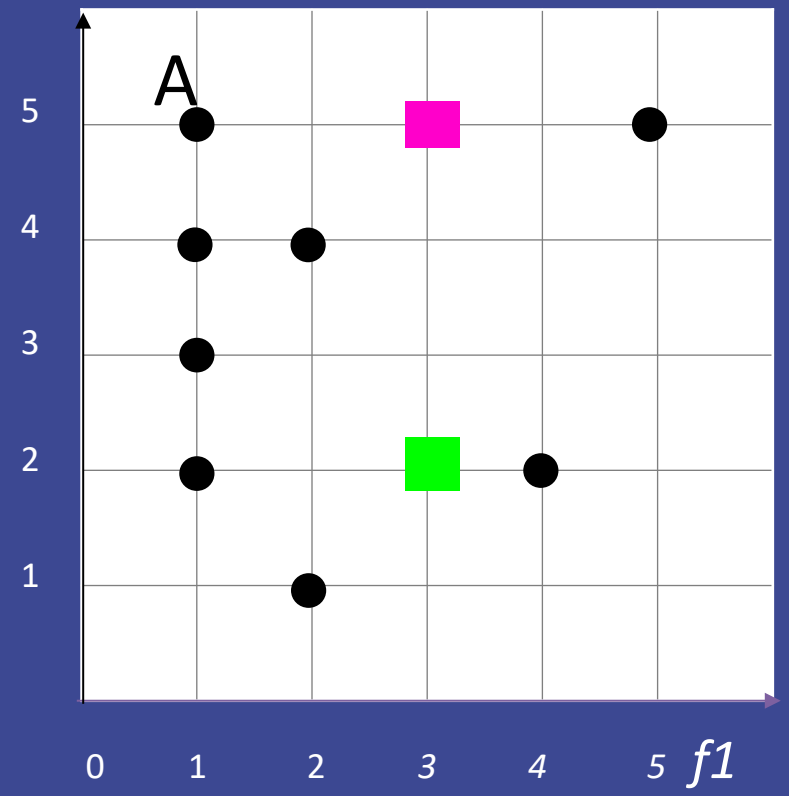
We start with the two cluster centers shown: pink (3, 5) and green (3, 2).



K-Means Clustering

Which cluster center is initially assigned to example A?

A. the green cluster center
B. the pink cluster center

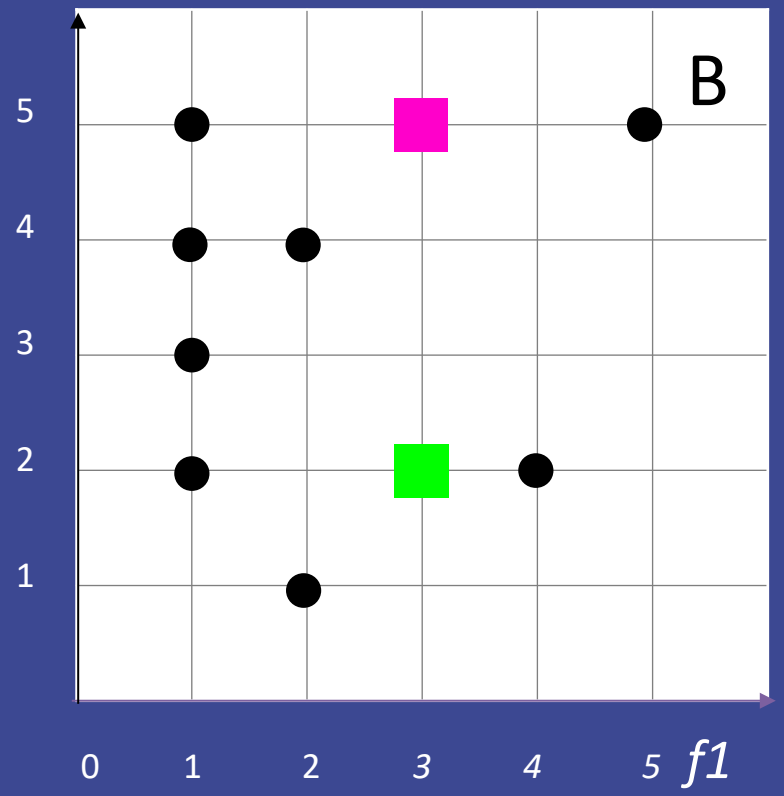


K-Means Clustering

Which cluster center is initially assigned to example B?

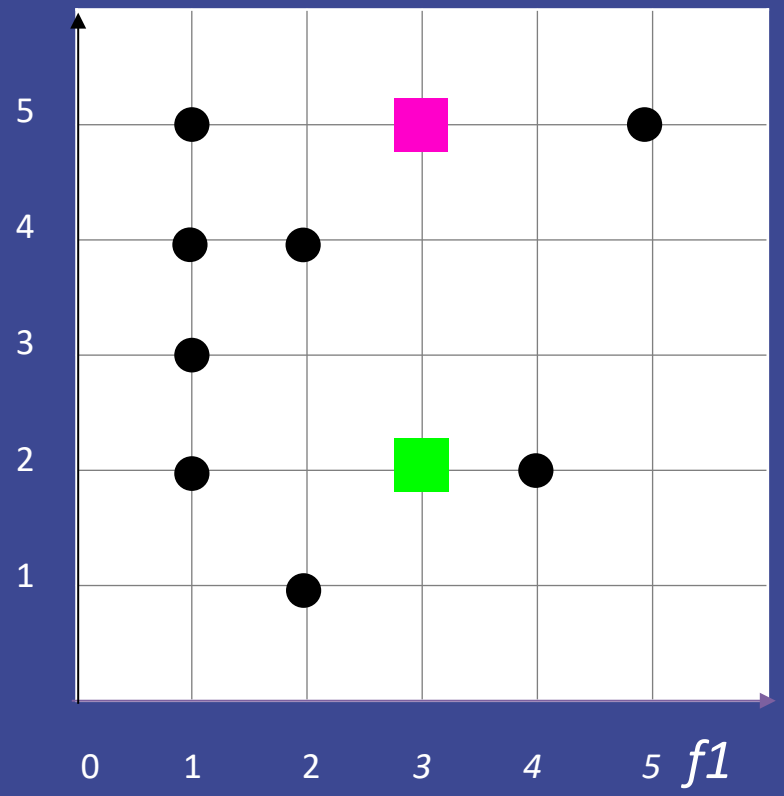
A. the green cluster center

B. the pink cluster center



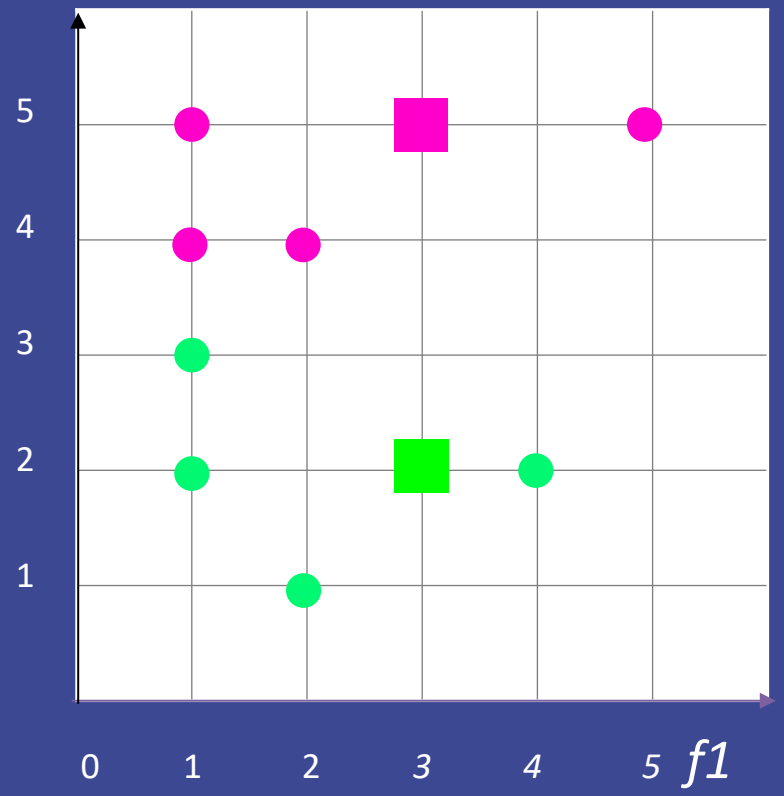
K-Means Clustering

How many examples are initially assigned to the pink cluster center?



K-Means Clustering

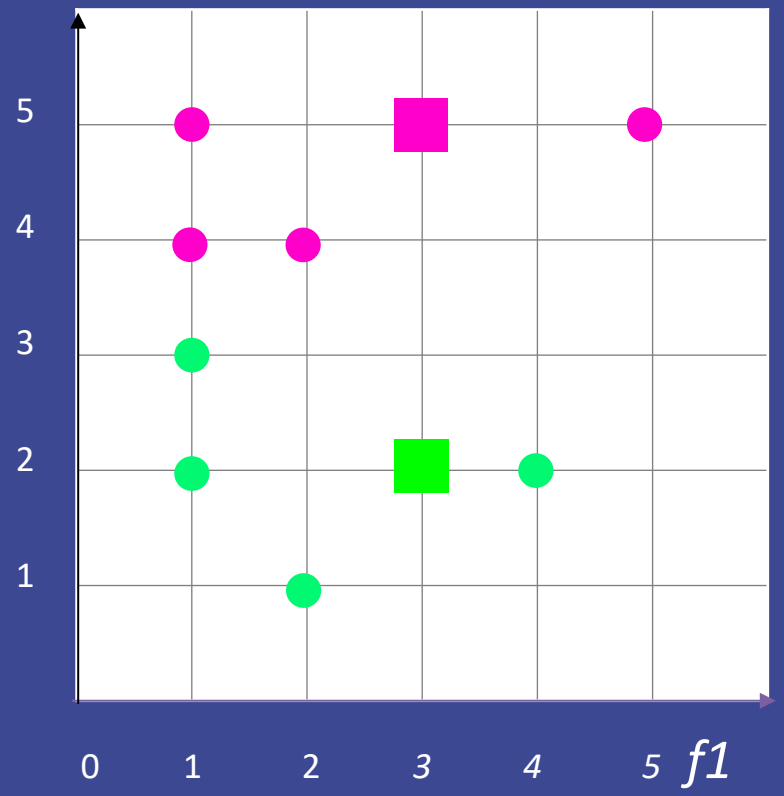
Phase 1: Assign each point to the closest mean/cluster center.



K-Means Clustering

Phase 2: Move the cluster centers to the mean of their assigned points.

What are the new coordinates of the cluster centers?

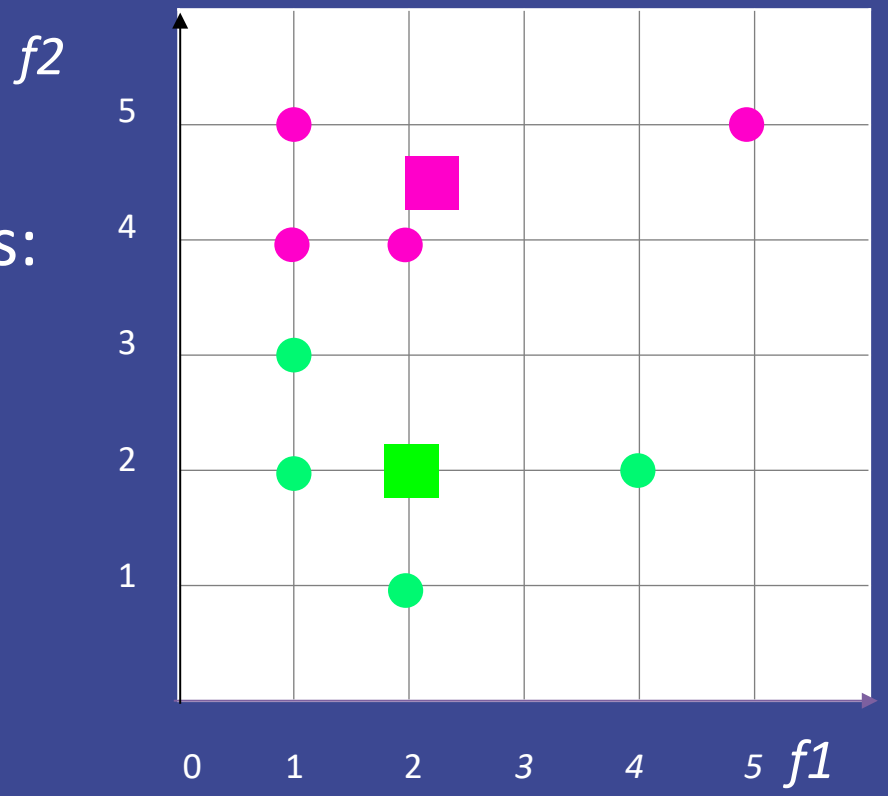


K-Means Clustering

To calculate the new position of the means:

- f1 = (1 + 1 + 2 + 4) / 4 = 2
  f2 = (2 + 3 + 1 + 2) / 4 = 2

- f1 = (1 + 1 + 2 + 5) /4 = 2.25
  f2 = (4 + 5 + 4 + 5) / 4 = 4.5

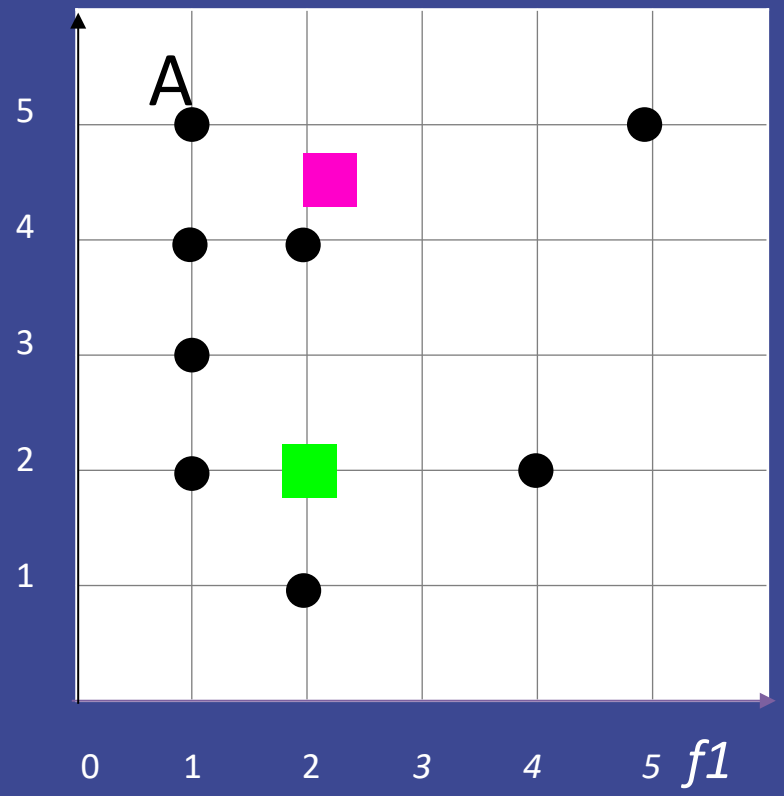


K-Means Clustering

Which cluster center will then assigned to example A?

A. the green cluster center

B. the pink cluster center

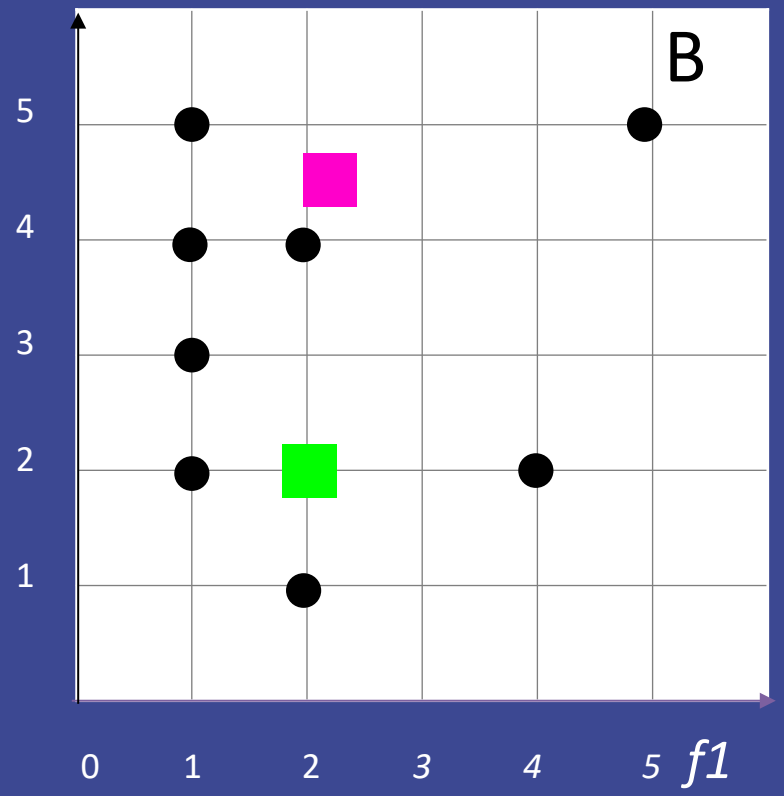


K-Means Clustering

Which cluster center will then be assigned to example B?

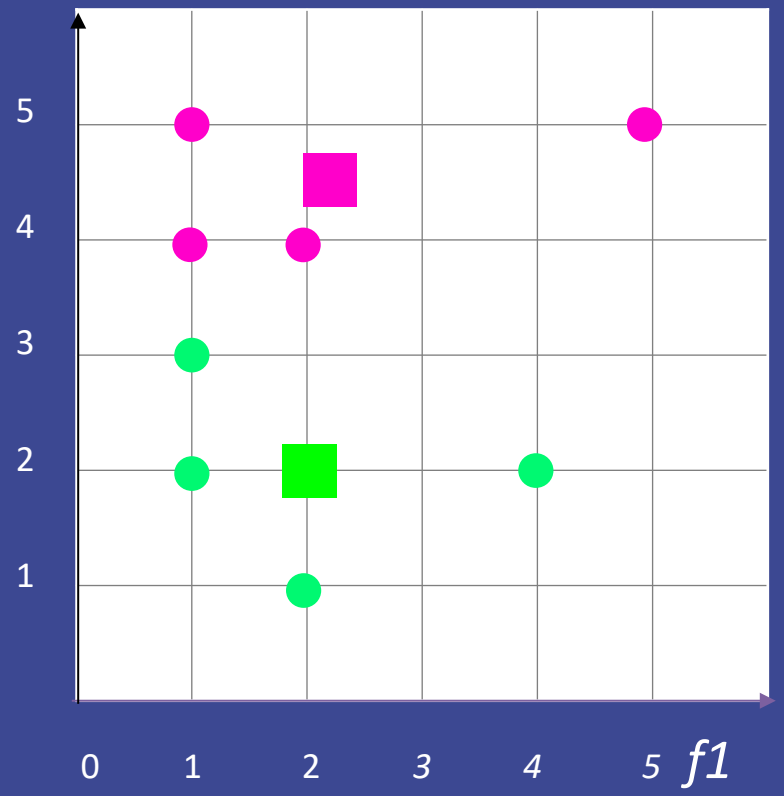
A. the green cluster center

B. the pink cluster center



K-Means Clustering

What are the new coordinates of the cluster centers?



Next

- QoW on Tuesday
- Homework 7 due December 3
- Reinforcement Learning